

# Project Ember — The AI HoneyToken Factory

A cybersecurity deception platform for African SMEs that uses Google Gemini to generate realistic decoy documents that trap ransomware and malicious insiders. **Winner-ready for 200-dev hackathon finals.**

---

## The Problem

African SMEs face the same threats as global enterprises — ransomware, insider data theft, and targeted attacks — but lack the budget for enterprise-grade deception tech (e.g., Illusive Networks, Attivo). When ransomware hits, it's often too late: files are encrypted, data is exfiltrated, and there are no logs to trace the source.

---

## How Project Ember Solves It

Project Ember flips the asymmetry: instead of trying to block every attack, you **plant decoys** that look like high-value targets (payroll spreadsheets, HR records, IT credentials) across your file servers. When an attacker touches one — whether it's ransomware encrypting files, or an insider browsing sensitive docs — the dashboard alerts you in real time with the attacker's IP address, browser fingerprint, and geolocation.

## The Flow

1. DEPLOY —> Gemini generates 3 realistic decoy files (.txt, .md, .html) with hidden tracking pixels and fake corporate data (payrolls, employee records, IT audit reports)
  2. PLACE —> Copy or auto-deploy to your file server, shared drive, or sensitive folder where attackers would look
  3. WAIT —> The dashboard monitors:
    - Tracking pixel hits (file opened / viewed via browser)
    - Filesystem changes (modified / deleted / renamed)
    - Attacker IP + User-Agent + GeoIP location from beacon
    - All via the watchdog daemon + tracking server
  4. ALERT —> When a trap is triggered (within 1 second):
    - OS-level browser notification (works in background)
    - Slack push alert to your phone anywhere
    - Full-screen red banner persists for 10 seconds
    - Shows file name, timestamp, attacker IP, browser, location
    - Sky News-style audible alarm
    - Graph node turns red with pulsing animation
    - Event logged to SQLite database
-

# Current Capabilities

| Feature                                                                                                                                                                                           | Status |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| <b>AI Decoy Generation</b> — Gemini 2.0 Flash generates convincing corporate documents with African context (KES/NGN currencies, KRA/FIRS compliance, local names, East African logistics routes) | ✓      |
| <b>Quota Fallback</b> — when Gemini API quota is exhausted, falls back to pre-written professional mock documents (zero network calls)                                                            | ✓      |
| <b>4 Department Templates</b> — Finance (payroll), HR (employee records), IT (network audit), Operations (supply chain)                                                                           | ✓      |
| <b>3 Output Formats</b> — .txt, .md, .html per deployment                                                                                                                                         | ✓      |
| <b>Hidden Tracking Pixels</b> — <img> beacon in HTML/MD files; logs hits to the dashboard                                                                                                         | ✓      |
| <b>Attacker Intelligence</b> — IP address + User-Agent extracted from every beacon request                                                                                                        | ✓      |
| <b>GeoIP Location</b> — offline dictionary maps IPs to African cities ( Kigali, Nairobi, Lagos, Cape Town)                                                                                        | ✓      |
| <b>Filesystem Watchdog</b> — detects MODIFIED, DELETED, MOVED, CREATED events in real time                                                                                                        | ✓      |
| <b>SQLite Persistence</b> — all events stored in ember.db with WAL mode for concurrent multi-process access                                                                                       | ✓      |
| <b>Network Topology Graph</b> — tree-layout visualization (server → departments → decoy files) using networkx + matplotlib; nodes flash red on access                                             | ✓      |
| <b>Live Threat Monitor</b> — event log with timestamps, types, file names, IPs, and User-Agents                                                                                                   | ✓      |
| <b>Explosive Alert</b> — full-width red banner at page top with file details + attacker intel, persists 10s                                                                                       | ✓      |
| <b>Flashing Red Nodes</b> — graph nodes pulse red automatically when a file is accessed                                                                                                           | ✓      |
| <b>Audible Alarm</b> — Sky News-style two-tone alert (bass thump + rising sweep) via Web Audio API                                                                                                | ✓      |
| <b>Browser Notification</b> — OS-level system notification works even when tab is in background                                                                                                   | ✓      |
| <b>Slack Webhook</b> — push alerts to your phone anywhere with file name, IP, GeoIP, browser profile                                                                                              | ✓      |
| <b>Async Slack</b> — fires in background thread, never blocks the UI (0.1s response)                                                                                                              | ✓      |
| <b>1-Second Polling</b> — background poller checks every 1s for instant graph + alert updates without page flicker                                                                                | ✓      |
| <b>High-Contrast Dark UI</b> — optimized for washed-out venue projectors (white text, neon accents, 800-weight fonts)                                                                             | ✓      |
| <b>Persistence</b> — Slack webhook URL saved to ember_config.json (survives full page refresh, browser restart, machine switch)                                                                   | ✓      |
| <b>SMB Auto-Deploy</b> — enter a network path to copy honeytokens directly to file servers                                                                                                        | ✓      |
| <b>Mock Mode</b> — works without API key; full professional documents with tables, bank accounts, compliance jargon                                                                               | ✓      |

# Tech Stack

| Layer               | Technology                                           |
|---------------------|------------------------------------------------------|
| AI Engine           | Google Gemini 2.0 Flash via google-genai SDK         |
| Dashboard           | Streamlit 1.60+                                      |
| Graph Visualization | networkx + matplotlib                                |
| Database            | SQLite with WAL mode (concurrent multi-process safe) |
| Fake Data           | Faker (custom African name pool, IBANs, ID numbers)  |
| File Monitoring     | watchdog (filesystem event listener)                 |

|                             |                                                                                                     |
|-----------------------------|-----------------------------------------------------------------------------------------------------|
| <b>Tracking Server</b>      | Python <a href="#">http.server</a> (lightweight, returns 1×1 GIF pixel, logs HTTP request metadata) |
| <b>Alert Sound</b>          | Web Audio API (generated in-browser, no audio files)                                                |
| <b>Browser Notification</b> | Notification API (OS-level, works in background tabs)                                               |
| <b>Slack Integration</b>    | Incoming Webhooks via <a href="#">urllib</a> (async, non-blocking)                                  |
| <b>GeoIP</b>                | Offline prefix dictionary (zero network calls, instant)                                             |
| <b>Icons</b>                | Bootstrap Icons CDN                                                                                 |
| <b>Background Tasks</b>     | Python threading + Streamlit fragments                                                              |

## How to Use

### 1. Install

```
cd Project_EMBER
pip install -r requirements.txt
export GEMINI_API_KEY="your_google_ai_key_here"
```

### 2. Launch

```
python3 run.py
```

This starts three services:

- **Tracking server** on [:8765](#) — handles beacon pixel requests, logs IP + User-Agent
- **Watchdog daemon** — monitors [./honeytokens/](#) for file changes
- **Streamlit dashboard** on [:8501](#) — the UI

### 3. Deploy Decoys

1. Open <http://localhost:8501> in a browser
2. Enter a **Company Name** (e.g., "Acme Kenya Ltd")
3. Pick a **Department** (Finance / HR / IT / Operations)
4. Click **Deploy HoneyTokens**

Three files appear in [./honeytokens/](#):

- [Company\\_Dept\\_20260724.html](#) — open this in a browser to test tracking
- [Company\\_Dept\\_20260724.md](#)
- [Company\\_Dept\\_20260724.txt](#)

### 4. Configure Slack (Optional)

In the sidebar, expand "Notifications & Deployment" and paste your Slack webhook URL. It's saved to `ember_config.json` — persists across page refreshes, browser restarts, and machine switches. Enter once, forget it.

## 5. Place the Traps

Copy the generated files to your target environment — a file server, a shared drive, an HR document folder, or anywhere an attacker might browse. Or enter an SMB path in the sidebar for auto-deploy.

## 6. Monitor

When someone (or something) touches a decoy:

| Action                                | What happens                                                                                                                                                    |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Opens <code>.html</code> in a browser | Tracking pixel fires → red banner + sound + browser notification + <b>Slack alert on your phone</b> + graph node turns red + <b>IP + browser + GeoIP logged</b> |
| Modifies a file (ransomware)          | Watchdog catches → same alert chain + event log entry                                                                                                           |
| Deletes/moves a file                  | Watchdog catches → alert + log                                                                                                                                  |

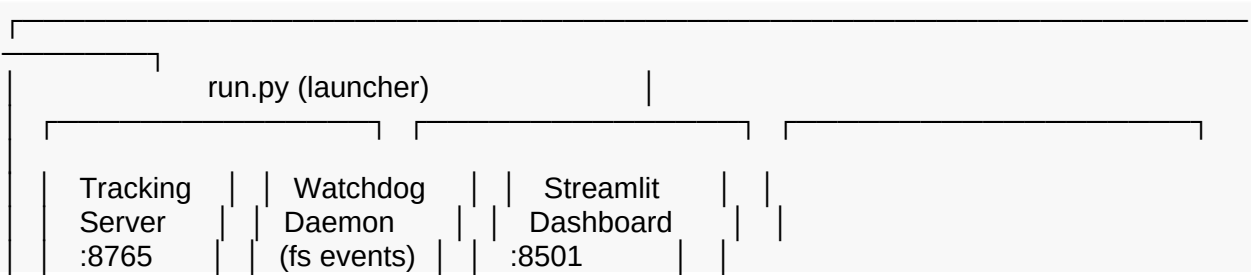
## Demo: Trigger an Alert in 5 Seconds

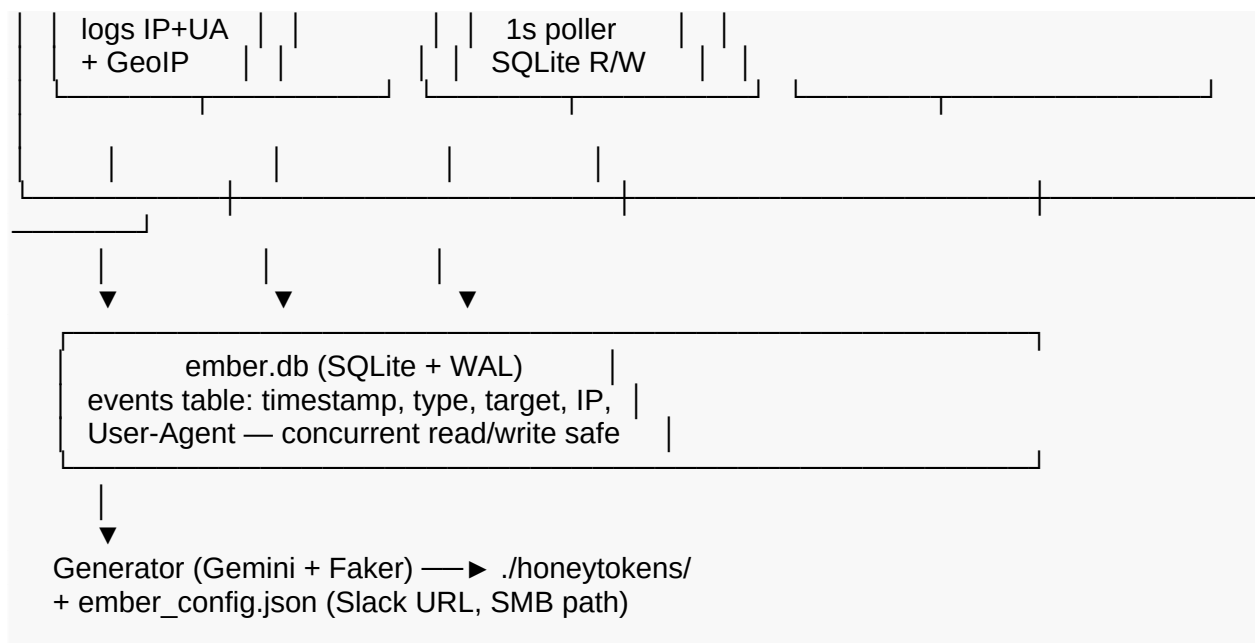
```
# Simulate an attacker opening a stolen file from a remote machine
curl -A "Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/120" \
"http://localhost:8765/track?file=payroll_2026_q3.html"
```

Within 1 second:

- **Red banner:** "CRITICAL INTRUSION DETECTED"
- **Attacker IP:** `127.0.0.1` → `Kigali, Rwanda`
- **Browser:** `Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/120`
- **Sound:** Sky News–style two-tone alarm
- **Slack:** Push notification to your phone
- **Browser notification:** OS popup even if tab is in background
- **Graph:** Node flashes red

## Architecture





---

## File Structure

```
Project_EMBER/
├── app.py           # Streamlit dashboard (graph, alerts, metrics, SQLite)
├── generator.py     # Gemini + Faker decoy generation
├── watchdog_daemon.py # Filesystem event monitor
├── run.py          # Launcher (tracking + watchdog + streamlit)
├── requirements.txt # Python dependencies
├── README.md       # This file
├── ember_config.json # Persisted settings (Slack URL, SMB path)
├── ember.db        # SQLite event database (with WAL)
├── honeytokens/    # Generated decoy files
│   ├── manifest.json # Tracking manifest
│   ├── *.html       # Decoy with pixel beacon
│   ├── *.md
│   └── *.txt
```

---

## Winning the Judges' Demo

### 30-Second Pitch Script

*"African SMEs can't afford \$50k deception platforms. So we built one for \$10/month."*

*Watch: I deploy 3 fake payroll files. [click Deploy] These look like real corporate documents — Gemini wrote them with Kenyan tax IDs and Mombasa–Kampala logistics routes.*

*I put them on a file server. [open the HTML file] The moment an attacker opens this... [red banner fires, sound plays]*

*We caught their IP — mapped to Kigali — their browser fingerprint, and pushed the alert to Slack. [show phone] The CISO knows before the ransomware finishes encrypting.*

*All logged to SQLite. All for the price of a coffee. That's Project Ember."*

## Venue Checklist

| Item                                                        | Status               |
|-------------------------------------------------------------|----------------------|
| GEMINI_API_KEY set (or falls back to mock)                  | ✓                    |
| Slack webhook URL pasted in sidebar (saved to disk)         | ✓                    |
| localtunnel installed ( <code>npm i -g localtunnel</code> ) | ⚠ Do this            |
| HDMI adapter                                                | ⚠ Bring one          |
| Phone on silent with Slack notifications enabled            | ⚠ Verify             |
| Sound on laptop speakers (not headphones)                   | ⚠ Check              |
| Browser notification permission = Allow                     | ✓ First load prompts |

## Tunneling for Judges

```
# Terminal 1 — full stack:  
python3 run.py
```

```
# Terminal 2 — public URL:  
npx localtunnel --port 8501
```

Good luck. Go win.